

Conversations & Memory

Keep continuity without confusing state, history, context, and storage

Source: <https://learn.microsoft.com/en-us/agent-framework/concepts/agents/conversations/session?tabs=python>

Sessions

- **AgentSession is the conversation state container used across agent runs**
- **What AgentSession contains:**
 - `session_id` — Local unique identifier for this session
 - `service_session_id` — Remote service session identifier, such as a conversation or response ID, when service-managed history is used
 - `state` — Mutable dictionary shared with context/history providers
- **Sessions are agent/service-specific** — Reusing a session with a different agent configuration or provider can lead to invalid context; restore a service-side session id only for the user or tenant that owns it

The lifecycle is explicit

```
from agent_framework import AgentSession

session = agent.create_session()
await agent.run("Remember this project.", session=session)
await agent.run("Which project?", session=session)

# Create a session from an existing service conversation id
existing = agent.get_session(service_session_id=owned_service_id)
# Serialize
payload = session.to_dict()
# Deserialize
resumed = AgentSession.from_dict(payload)
```

D E M O

~5 min

Serialize, kill the process, restore the session

Run the previous slide's exact code live: create a session, run two turns, serialize with `session.to_dict()`, simulate a fresh process, restore with `AgentSession.from_dict()`, and continue the same conversation — proving the session container, not the process, carries continuity.

Runbook: [demos/day3/module-1-demo-1-serialize-restore.md](#)

Context Providers

- Context providers run at every invocation to add context before execution and process data after
- This includes tools for progressive loading
- Configure providers through `context_providers=[...]`

```
from agent_framework import Agent, InMemoryHistoryProvider
from agent_framework.openai import OpenAIChatClient

agent = Agent(
    client=OpenAIChatClient(),
    name="MemoryBot",
    instructions="You are a helpful assistant.",
    context_providers=[InMemoryHistoryProvider("memory", load_messages=True)],
)

session = agent.create_session()
await agent.run("Remember that I prefer vegetarian food.", session=session)
```

Custom Context Providers

- Use custom context providers when you need to inject dynamic instructions/messages/tools or extract state after runs.

```
from typing import Any
from agent_framework import AgentSession, ContextProvider, SessionContext

class UserPreferenceProvider(ContextProvider):
    def __init__(self) -> None:
        super().__init__("user-preferences")

    async def before_run(
        self, *,
        agent: Any,
        session: AgentSession,
        context: SessionContext,
        state: dict[str, Any],
    ) -> None:
        if favorite := state.get("favorite_food"):
            context.extend_instructions(self.source_id, f"User's favorite food is {favorite}.")

    async def after_run(
        self, *,
        agent: Any,
        session: AgentSession,
```

Storage

Mode	What is stored	Typical usage
Local session state	Full chat history in <code>AgentSession.state</code> (for example via <code>InMemoryHistoryProvider</code>)	Services that don't require server-side conversation persistence
Service-managed storage	Conversation state in the service; <code>AgentSession.service_session_id</code> points to it	Services with native persistent conversation support

- Two storage modes: Local and service managed
- Local storage can be reduced, service managed reduction is service specific
- Use more than one provider for auditing (`load_messages=False`, `store_context_messages=True`)
- Third party/custom storage providers for alternative storage options

ContextProvider wraps each invocation

- **1. Caller input** — Your app passes messages and session to `agent.run(...)`
- **2. before_run** — Provider can call `context.extend_instructions(source_id, ...)`, `context.extend_tools(source_id, ...)`, or `context.extend_middleware(source_id, ...)` to enrich this one invocation
- **3. Agent execution** — Model and tools run with the enriched instructions, tools, and middleware
- **4. after_run** — Provider reads `context.input_messages` (and the response) to decide what state to persist for next time

One provider loads messages

- **Loader** — Exactly one HistoryProvider is configured with `load_messages=True`
- **Observers** — Additional history providers use `load_messages=False`; set `store_context_messages=True` so they still capture context from other providers alongside input/output
- **Why one loader** — Two loaders can duplicate history and distort what the model sees
- **Typical use** — An audit/eval history provider that observes without affecting primary history loading

Durability is an application choice

- **Store messages under a session-scoped key** — one session's history shouldn't leak into another's
- **Keep returned history within model context limits** — apply a compaction strategy (Module 3) or a reducer if it grows too large
- **Persist provider-specific identifiers in the session state** — not as a side-channel your app has to track by hand
- **Per-service-call persistence is opt-in** — `require_per_service_call_history_persistence=True` runs local history providers around every model call inside a tool loop, not just once after the full `agent.run()` completes

A service conversation ID is not authorization

- **Not proof of ownership** — "A continuation ID identifies a session to resume; it doesn't prove that the caller owns that session."
- **1. Authenticate the caller**
- **2. Authorize the caller to access the referenced state**
- **3. Partition durable state by the authenticated tenant, user, or workspace**
- **4. Persist session and checkpoint state only after the run or stream has completed**

Decision guide

- **Short-lived local chat** — InMemoryHistoryProvider; verify process loss is acceptable
- **Server-side conversation** — Service-managed session; verify the service and provider both support it
- **Dynamic memory or policy** — ContextProvider; verify before_run/after_run state is session-scoped
- **Restart durability** — App-owned persistence, or a self-host SessionStore; verify encryption, retention, and concurrency
- **Multi-tenant resume** — A trusted ownership mapping; verify raw service IDs are never treated as authorization

Takeaways



You create one `AgentSession` and pass `session=` on every turn.



You use `get_session(...)` only with an authorized, owned service conversation ID.



You separate session state, message history, context enrichment, and self-host storage.



You configure only one message-loading `HistoryProvider`.



You persist and secure continuity at your application boundary.